
1. ¿Por qué un diagrama de Gantt?

1.1. Más que un cronograma: el Gantt como mapa cognitivo del proyecto

Durante mis años en la industria del software, he aprendido que un proyecto sin un cronograma visual es como navegar sin brújula. El diagrama de Gantt, cuyo origen se remonta a las técnicas de planificación de Henry L. Gantt a principios del siglo XX, ha trascendido su formato analógico para convertirse en el **estándar de facto de la comunicación de plazos**. Pero no se trata únicamente de dibujar barras en un calendario; el Gantt moderno es un modelo de la **dimensión temporal** del proyecto que integra tareas, dependencias, hitos, recursos y restricciones de calendario en una vista unificada.

Cada barra horizontal representa una actividad con un inicio y un fin definidos. La longitud de la barra codifica la duración, y su posición en el tiempo refleja las restricciones de precedencia que establecimos en la red de dependencias. Cuando un ingeniero de sistemas observa un Gantt bien construido, puede responder de inmediato preguntas críticas: ¿qué tarea debería estar en ejecución hoy? ¿cuál es el impacto de un retraso en la actividad A sobre la entrega final? ¿qué equipos están sobrecargados en una semana determinada?

En la práctica, el diagrama de Gantt no es un mero producto, sino un **proceso de construcción** que ya desde el inicio obliga al equipo a reflexionar sobre:

- la descomposición del alcance (¿están todas las tareas necesarias?),
- la secuencia lógica (¿realmente C no puede empezar hasta que A termine?),
- y los supuestos temporales (¿por qué estimamos 6 días para la actividad A?).

1.2. La sinergia con PERT/CPM: de la incertidumbre al plan ejecutable

En clase hemos revisado un caso concreto, allí aplicamos el método PERT (Program Evaluation and Review Technique) para calcular tiempos esperados a partir de estimaciones optimistas, más probables y pesimistas, obteniendo una red con valores como 6 días para la actividad A, 2 para B, 3 para C, etc. La técnica CPM (Critical Path Method) nos reveló que la ruta A-C-E-H-I-J era la que dictaba la duración total del proyecto (20 días laborables) y que otras tareas tenían holgura (por ejemplo, B, D, F, G podían demorarse sin afectar la fecha final, salvo que se acumularan retrasos).

Ahora bien, ¿qué papel juega el diagrama de Gantt en este contexto? Mientras que PERT/CPM se enfoca en **analizar la incertidumbre y las holguras** mediante grafos dirigidos, el Gantt toma ese esqueleto lógico y lo proyecta en un calendario real. PERT nos dice *"la ruta crítica es A-C-E-H-I-J y debemos vigilarla especialmente"*; el Gantt nos muestra *día a día* cuándo cada una de esas actividades está programada, facilitando la supervisión diaria, la asignación de recursos y la comunicación con stakeholders no técnicos.

Por lo tanto, no son herramientas rivales sino complementarias. Yo suelo usar PERT en la fase de planificación temprana para detectar cuellos de botella y estimar probabilidades de cumplimiento (usando la varianza), y luego vuelco esa estructura en un Gantt que se convierte en el tablero de control durante la ejecución. De hecho, en el documento [PlantUML_Language_Reference_Guide_es.pdf] se resalta que los

diagramas de Gantt son particularmente útiles para la gestión de proyectos y que PlantUML permite definirlos con un enfoque basado en texto (páginas 341-380), lo que facilita su evolución continua.

1.3. Dimensiones que el Gantt captura y que el código nos da gratis

A diferencia de un diagrama estático dibujado en una pizarra, un Gantt automatizado con PlantUML permite manejar múltiples capas de información:

1. **Dependencias finitas y complejas:** gracias a la sintaxis `starts at [Tarea]'s end`, `ends at`, o `happens at`, podemos modelar encadenamientos que reflejan la lógica real del desarrollo (por ejemplo, que la tarea H requiere tanto E como F terminadas, un típico "join" que en la red PERT sería un nodo de unión).
2. **Hitos:** Representan eventos de duración cero. En nuestro ejercicio, definimos un hito `[Integración GH]` que señala el momento en que ambas ramas del proyecto convergen. Los hitos son esenciales para la gobernanza: informes de estado, revisiones de diseño, o fechas de entrega contractual.
3. **Asignación de recursos:** La notación `on {Alice}` y `on {Bob:50%}` permite visualizar cargas de trabajo. Incluso se pueden marcar días en los que un recurso está inactivo (`{Alice} is off on ...`), algo fundamental para evitar sobreasignaciones.
4. **Calendario realista:** Podemos declarar días cerrados (`saturday are closed`, `sunday are closed`, fechas específicas como `2026-05-15 is closed`). Esto ajusta automáticamente la duración en días hábiles, reflejando la realidad operativa de la organización. Recordemos que en PERT hablamos de días ideales; el Gantt los aterriza.
5. **Seguimiento del avance:** Con directivas como `is 40% completed` y con la personalización de colores para tareas completadas, el Gantt se convierte en un tablero de control vivo que puede generar reportes de estado al instante.

1.4. Beneficios estratégicos para el ingeniero de software que actúa como líder técnico

Más allá de la técnica, adoptar el Gantt como herramienta de gestión en una cultura de ingeniería produce beneficios concretos:

- **Comunicación efectiva con stakeholders no técnicos:** Un Gantt bien diseñado es autoexplicativo. Muestra fechas clave, duraciones y dependencias sin necesidad de explicar la red PERT subyacente. La leyenda puede incluir la ruta crítica resaltada (como haremos en el ejercicio con `note bottom`).
- **Detección temprana de desviaciones:** Al comparar la línea base (el plan original) con el progreso real, cualquier desplazamiento de una barra crítica dispara alarmas. En proyectos de software, donde los cambios de requisitos son frecuentes, el Gantt dinámico permite replanificar rápidamente.
- **Integración con metodologías ágiles e híbridas:** Aunque el Gantt suele asociarse a cascada, en mis equipos lo usamos para planificar *releases* y *sprints* a nivel macro, dejando los tableros Kanban

para el detalle diario. La vista de Gantt complementa la retrospectiva del sprint al mostrar el avance acumulado.

- **Versiónamiento y colaboración:** Al estar definido como código, el Gantt se puede almacenar en Git junto al código fuente. Cualquier modificación en las fechas o dependencias queda registrada en el historial de commits, y los cambios se pueden revisar en pull requests, como cualquier otro artefacto de software. Esto elimina para siempre el "Gantt_diagram_final_v3_bueno.xlsx" que nadie actualiza.

1.5. Un vistazo al futuro que construiremos

En esta clase, partiendo del análisis PERT que ya dominamos, construiremos paso a paso un diagrama de Gantt completamente funcional en PlantUML, aplicando los conceptos anteriores y añadiendo capas de sofisticación. Empezaremos con la estructura básica de tareas y duraciones, añadiremos las dependencias, insertaremos hitos de integración, personalizaremos colores para destacar la ruta crítica y, finalmente, lo pondremos bajo control de versiones. Al final, habrán experimentado en carne propia por qué el Gantt es la herramienta de comunicación temporal por excelencia en la ingeniería del software moderna, y cómo su generación automatizada con PlantUML lo convierte en un activo vivo del proyecto, no en un simple dibujo estático.

2. Elementos fundamentales del Gantt: los átomos de la planificación temporal

Si observan con atención cualquier cronograma de proyecto, verán que su estructura no es caótica: está compuesta de un conjunto pequeño pero potente de elementos que, combinados, son capaces de modelar la complejidad temporal de sistemas de software de cualquier escala. Dominar cada uno de estos elementos es lo que separa a un Project Manager que simplemente dibuja barras de quien realmente **diseña y controla la ejecución** del proyecto. Vamos a desmenuzarlos uno por uno, conectándolos con los documentos de estudio y con el proyecto modelo que analizamos (actividades A–J).

2.1. Tareas y duraciones: los bloques básicos

Las tareas (o actividades) son las unidades de trabajo que consumen tiempo y recursos, y que en conjunto definen el alcance total. En nuestro caso, descompusimos el proyecto en diez actividades: A hasta J, cada una con una duración estimada (6, 2, 3, 3, 4, 3, 6, 4, 1, 2 días respectivamente). En PlantUML, la definición más simple de una tarea es mediante la notación `[Nombre] requires Duración`, como vemos en la guía (página 341). También podemos usar una sintaxis más compacta si combinamos inicio y fin: `[Prototype design] starts 2026-05-05 and requires 15 days`.

Pero detrás de esa sencillez sintáctica hay decisiones profundas. La duración no es un número mágico: en la metodología PERT, como vimos, se calcula a partir de tres estimaciones (optimista, más probable, pesimista) con la fórmula $(a + 4m + b)/6$. Así obtuvimos que la actividad A, que en el mejor de los casos podría tomar 3 días y en el peor 11, se estabiliza en 6 días como tiempo esperado. Esa duración ya incorpora una ponderación del riesgo. Sin embargo, al planificar en un Gantt real, la duración se expresa

en días laborables de calendario, no en días ideales. Si nuestro proyecto trabaja de lunes a viernes, 6 días equivalen a una semana y un día. Esto es crítico: una tarea de 6 días que empieza un lunes terminará el lunes siguiente, no el domingo (porque sábado y domingo están cerrados).

Un error común en ingenieros novatos es confundir **duración** con **esfuerzo**. La duración es el tiempo calendario que ocupa la actividad (6 días), mientras que el esfuerzo son las horas-persona necesarias (por ejemplo, 48 horas si una persona dedica 8h/día). Si asigno dos personas a tiempo completo, la duración podría reducirse a 3 días, pero el esfuerzo sigue siendo 48 horas. En el Gantt, la duración se traduce directamente en la longitud de la barra, y la asignación de recursos (que veremos en el punto 2.4) puede influir en ella si se decide comprimir el cronograma.

2.2. Dependencias: el orden invisible que gobierna el flujo

Si las tareas fueran bloques aislados, podríamos ejecutarlas todas en paralelo y terminar en el tiempo de la más larga. La realidad es muy distinta: las tareas se relacionan entre sí mediante dependencias que imponen un orden estricto o flexible. En nuestro proyecto, la actividad C (3 días) no puede comenzar hasta que A (6 días) haya terminado completamente. Esto es una dependencia **fin-inicio** (Finish-to-Start o FS), la más común. Pero existen otros tres tipos en la gestión profesional:

- **Inicio-Inicio (SS)**: Dos tareas deben comenzar al mismo tiempo o una puede comenzar cuando la otra comienza (ejemplo: "Codificar" y "Escribir pruebas unitarias" pueden iniciar juntas).
- **Fin-Fin (FF)**: Dos tareas deben finalizar juntas (ejemplo: "Elaborar manual de usuario" no puede terminar hasta que "Desarrollo de funcionalidad" termine, porque el manual debe reflejar la funcionalidad completa).
- **Inicio-Fin (SF)**: Una tarea no puede finalizar hasta que otra comience (poco común; ejemplo: el turno de noche no puede terminar hasta que el turno de día comience).

PlantUML simplifica modelando principalmente FS con la cláusula **starts at [Tarea]'s end**, como usaremos para C, D, E, etc. Pero también podemos modelar dependencias más sutiles usando hitos o **happens at**. En el mundo real, una dependencia no es solo una restricción técnica; puede ser **contractual** (un proveedor entrega un componente), **normativa** (una auditoría debe ocurrir antes del despliegue), o **estratégica** (el CEO quiere revisar antes de continuar). Como PM, debo negociar y documentar cada dependencia, porque los retrasos se propagan a lo largo de la cadena. En el análisis CPM, la dependencia entre A y C crea la ruta crítica; si A se retrasa 2 días, todo el proyecto se retrasa 2 días, a menos que podamos acelerar C, E, H, I o J (lo cual suele ser costoso).

La guía de PlantUML nos ofrece una sintaxis expresiva para dependencias (páginas 342-344): **[T2] starts at [T1]'s end**, **[T2] starts 3 days after [T1]'s end**, incluso **[T2] starts at [T1]'s start**. Esto permite modelar solapamientos y retardos intencionados. En nuestro ejercicio, para la tarea H que necesita que E y F terminen, emplearemos dos líneas de dependencia, creando un punto de sincronización. Este es el equivalente gráfico de un nodo de unión en la red PERT, y visualmente se traduce en que la barra de H no puede comenzar hasta que ambas barras predecesoras hayan alcanzado su extremo derecho.

2.3. Hitos: faros en el calendario

Un hito es un evento de duración cero que marca un punto significativo en el proyecto: el fin de una fase, una decisión de financiamiento, la aprobación de un diseño, o simplemente un momento de

sincronización. En nuestra planificación, identificamos la convergencia de las ramas que vienen de G y H, y creamos el hito `[Integración GH]` para señalar el momento exacto en que ambas rutas se han completado y se puede proceder con I.

Los hitos cumplen funciones esenciales:

- **Gobernanza:** Los comités directivos revisan el proyecto en los hitos, no en tareas intermedias. Si defino un hito "Revisión de arquitectura" al terminar E, puedo condicionar la continuación del proyecto a una aprobación formal.
- **Medición de avance:** En metodologías de Valor Ganado, los hitos se utilizan como puntos de control objetivos. Un hito se cumple o no; no hay ambigüedad. Esto es mucho más confiable que porcentajes de avance subjetivos.
- **Sincronización de equipos:** En proyectos multidisciplinarios, un hito como "Especificación congelada" alinea a desarrollo, testing y documentación, indicando que a partir de ese momento los cambios requieren un proceso formal.

En PlantUML, los hitos se declaran con el verbo `happens` (página 346): `[Prototype completed] happens 2026-05-10` (fecha absoluta) o `[MaxTaskEnd] happens at [T1]'s end` (relativa). La notación `happens at` es especialmente útil cuando un hito depende del fin de **varias** tareas, como haremos con `[Integración GH] happens at [H]'s end` y luego forzamos que coincida con el fin de G también. Visualmente, los hitos se dibujan como un diamante o una barra muy delgada, destacando en el calendario.

2.4. Recursos: el combustible humano del proyecto

Ninguna tarea se ejecuta sola. Detrás de cada barra hay personas, equipos o incluso máquinas que realizan el trabajo. Asignar recursos a las tareas en el Gantt no es opcional: es la clave para detectar **sobreasignaciones, conflictos de disponibilidad y cuellos de botella** que ninguna dependencia lógica reflejaría.

Imaginemos que en nuestro proyecto, Alice es la líder técnica y Bob es el desarrollador senior. Alice ejecuta A, C, E; Bob ejecuta B, D, F, G; ambos colaboran en H, I, J. Si simplemente dibujamos las barras, parecería que las tareas de Bob están bien distribuidas, pero al asignar `on {Alice}` y `on {Bob}`, el Gantt (o herramientas de gestión) puede mostrar que Bob tiene demasiadas tareas en paralelo. Por ejemplo, B y D no deberían solaparse con F y G si Bob está al 100% en cada una; de lo contrario, tendríamos que ajustar duraciones o secuenciarlas explícitamente. PlantUML permite indicar `on {Bob:50%}` para señalar que Bob dedica medio tiempo, o incluso `{Bob} is off on 2026-05-10 to 2026-05-12` para marcar vacaciones o días de formación.

Además, la guía de PlantUML introduce la capacidad de ocultar recursos (`hide resources names`, `hide resources footbox`, página 359-360) según la audiencia. Para el equipo técnico, mostramos los nombres; para un cliente, quizá solo mostramos las tareas para evitar exponer información de personal. Esta flexibilidad es invaluable.

La gestión de recursos en proyectos de software suele ser compleja: un mismo desarrollador puede estar asignado a múltiples proyectos, y su disponibilidad real dicta cuánto puede avanzar. Como PM, debo negociar con los líderes funcionales para asegurar que los recursos críticos (como Alice, que está en la ruta

crítica) tengan dedicación exclusiva durante sus tareas, o de lo contrario la duración de la ruta crítica se alargará peligrosamente.

2.5. Calendario del proyecto: el tiempo no es continuo

El calendario es el lienzo sobre el que pintamos las barras. No podemos ignorar fines de semana, feriados, periodos de cierre de la empresa o mantenimiento planificado. En el ejemplo calculamos la duración en días ideales (20 días), pero en un Gantt real debemos especificar explícitamente qué días son laborables.

PlantUML ofrece un control fino del calendario (sección 16.10 y 16.11, páginas 347-348):

- `Project starts the 20th of september 2026` establece el inicio del proyecto.
- `saturday are closed` y `sunday are closed` definen la semana laboral estándar de lunes a viernes.
- `2026-05-15 is closed` para feriados puntuales.
- Incluso se pueden definir periodos abiertos o cerrados con rangos: `2026-05-20 to 2026-05-22 is closed`.

¿Qué sucede si no definimos el calendario? PlantUML asume que todos los días son laborables, incluyendo sábados y domingos. Como resultado, una tarea de 6 días que comienza un viernes terminaría el miércoles siguiente (contando sábado y domingo), lo que no refleja la realidad. Al activar `saturday are closed` y `sunday are closed`, automáticamente los 6 días se extienden sobre el eje temporal para ocupar solo días hábiles. Esto es fundamental para que el Gantt sea un **plan ejecutable**, no una abstracción.

Además, en proyectos internacionales, los calendarios pueden variar por ubicación. Un equipo en India tiene diferentes feriados que uno en Chile. Aunque PlantUML no maneja múltiples calendarios directamente, podemos segmentar el Gantt en fases y usar diferentes archivos o diagramas para cada equipo, o simplemente documentar las diferencias en notas.

La guía también explica cómo cambiar la escala del calendario (páginas 348-350) con `printscales weekly`, `projectscale monthly`, etc. Para nuestro proyecto de 20 días, una escala diaria es apropiada, pero en un proyecto de 2 años usaríamos escala mensual o trimestral para que el diagrama sea legible. Esto es un detalle de visualización que como PM debo ajustar según la audiencia: un comité directivo prefiere una vista trimestral, mientras que el equipo de desarrollo necesita el detalle diario.

2.6. Interdependencia de los elementos: la visión sistémica

Estos cinco elementos —tareas, dependencias, hitos, recursos y calendario— no funcionan aislados. Forman un sistema donde un cambio en uno se propaga a los demás. Si modifico la duración de C, se desplazan las fechas de inicio de E y F, lo que puede hacer que H ya no pueda contar con Bob porque en ese nuevo periodo Bob está de vacaciones (restricción de recurso). Si añado un feriado inesperado, las duraciones se alargan automáticamente, y posiblemente la holgura de B o D desaparezca, convirtiéndolas en parte de una nueva ruta crítica. Como ingeniero de software y futuro líder técnico, deben entrenar el ojo para ver estas interconexiones y usar el Gantt como un **modelo dinámico** que se recalcula y se simula mentalmente antes de comprometerse con una fecha de entrega.

En la siguiente sección, tomaremos todos estos conceptos y los materializaremos en código PlantUML, construyendo el Gantt para el proyecto de 10 actividades. Verán cómo la sintaxis declarativa permite capturar toda esta complejidad de manera concisa, y cómo el resultado visual refuerza la comprensión del plan. La teoría sin práctica es estéril; la práctica sin teoría es ciega. Ahora estamos listos para la práctica informada.

3. Manos a la obra: del análisis PERT al Gantt ejecutable

Después de haber asimilado la teoría de los elementos fundamentales, ha llegado la hora de ponerla en práctica con el proyecto que hemos estado analizando. En esta fase, recorreremos el proceso completo de transformación de los datos provenientes del análisis PERT en un diagrama de Gantt funcional y enriquecido, utilizando la sintaxis declarativa de PlantUML. La meta no es solo obtener una imagen bonita, sino entender cada decisión de diseño que como Project Manager debo tomar para que el cronograma refleje fielmente la realidad operativa y sirva como herramienta de control diario.

3.1. El punto de partida: la tabla PERT y la redde precedencias

Recordemos las lecciones anteriores, tras calcular los tiempos estimados y la varianza para cada actividad, obtuvimos la siguiente tabla consolidada que relaciona actividades, duraciones y predecesoras inmediatas:

Actividad	Duración (días)	Predecesoras
A	6	–
B	2	–
C	3	A
D	3	B
E	4	C
F	3	C
G	6	D
H	4	E, F
I	1	G, H
J	2	I

Esta tabla es el esqueleto lógico de nuestro proyecto. La actividad H, por ejemplo, solo puede comenzar cuando tanto E como F hayan terminado. La actividad I exige que G y H estén completas, y J depende exclusivamente de I. La ruta crítica calculada mediante CPM fue **A→C→E→H→I→J**, con una duración total de 20 días laborables. Otras actividades como B, D, F, G poseen holgura, lo que significa que sus fechas de inicio pueden variar dentro de ciertos márgenes sin afectar la fecha final del proyecto.

Sin embargo, esta tabla y la red PERT, aunque poderosas para el análisis, no responden preguntas cotidianas del equipo: "¿Qué tengo que hacer mañana? ¿Está Bob disponible para ayudar en la tarea H la próxima semana? ¿El cliente verá algún avance el viernes?". Para contestar a eso necesitamos proyectar esta red lógica sobre un calendario real y añadir las capas de recursos e hitos. Aquí es donde entra el Gantt.

3.2. La gran decisión: granularidad de las tareas en el Gantt

Antes de escribir una sola línea de código PlantUML, debo preguntarme: ¿el nivel de descomposición de la tabla PERT es adecuado para el Gantt de seguimiento? En nuestro caso, diez tareas son perfectamente manejables para un proyecto de 20 días, pero en la práctica industrial, un proyecto de meses puede tener cientos de tareas. Como PM, debo encontrar un equilibrio entre la visibilidad y la simplicidad. Divido demasiado y el diagrama se vuelve ilegible; agrupo demasiado y pierdo control sobre los detalles.

Para este ejercicio, mantendremos las diez tareas originales porque nos permiten ilustrar todos los conceptos sin abrumar. Sin embargo, quiero enfatizar que en equipos ágiles modernos, estas tareas podrían corresponder a *épicas* o *features*, y las tareas hijas se gestionarían en un backlog de sprint. El Gantt actuaría como la vista macro, y los tableros Kanban como la vista micro. Esta estratificación es una buena práctica que recomiendo adoptar.

3.3. Traduciendo dependencias a sintaxis PlantUML

El siguiente paso es convertir las precedencias de la tabla a sentencias comprensibles por PlantUML. La guía nos ofrece dos formas principales: `starts at [Tarea]'s end` para dependencias fin-inicio simples, y el uso de hitos para coordinar múltiples predecesoras.

Para actividades con una única predecesora, la traducción es directa:

- C depende de A → `[C] starts at [A]'s end`
- D depende de B → `[D] starts at [B]'s end`
- E depende de C → `[E] starts at [C]'s end`
- F depende de C → `[F] starts at [C]'s end`
- G depende de D → `[G] starts at [D]'s end`
- I depende de G y H → necesitamos una construcción especial.
- J depende de I → `[J] starts at [I]'s end`

Para H, que tiene dos predecesoras (E y F), la solución más limpia es escribir dos líneas de dependencia: `[H] starts at [E]'s end` y `[H] starts at [F]'s end`. PlantUML se encargará de situar H en la fecha más tardía entre los fines de E y F (que en nuestro caso será el fin de E, pues E es más larga que F y ambas comienzan juntas tras C). Esto modela correctamente la condición de que H solo arranca cuando ambas han terminado.

La situación de I es un poco más compleja porque sus predecesoras G y H terminan en momentos distintos: G demanda 6 días desde el inicio de D, mientras que H toma 4 días tras el fin de E. Dado que D y B están en una rama paralela con cierta holgura, es posible que G termine antes o después que H, dependiendo de cuándo comience B. Para garantizar la sincronización, introduciremos un hito `[Integración GH]` que ocurra exactamente en `[H]'s end`, y además forzaremos que I espere a que G

también haya terminado con `[I] starts at [G]'s end`. De esta manera, I empezará en el máximo de los dos finales.

3.4. Aterrizando en un calendario: inicio del proyecto y días hábiles

Nuestra tabla PERT maneja días abstractos. Para hacerla ejecutable, debemos anclar el proyecto a una fecha real. Propondré como ejemplo `Project starts 2026-05-05`, un martes. A partir de ahí y con la directiva `saturday are closed` y `sunday are closed`, la duración de 6 días de A se traducirá en una barra que va del martes 5 al martes 12 (ya que no se cuentan sábado 9 ni domingo 10). Este simple ajuste de calendario ya modifica las expectativas respecto a los 6 días ideales.

Además, puedo añadir días cerrados puntuales, por ejemplo `2026-05-15 is closed`, para simular un feriado. Esto extenderá automáticamente cualquier tarea que estuviera activa en esa fecha, demostrando cómo el Gantt refleja la realidad de forma dinámica.

Una decisión menos obvia es la escala visual. Dado que nuestro proyecto abarca aproximadamente un mes, la escala predeterminada diaria (`printscale daily`) es la adecuada. Si estuviéramos planeando sobre un año, usaría `projectscale monthly` para no saturar de rayitas verticales el diagrama. La guía (páginas 348-350) es muy clara al respecto: la escala afecta la legibilidad, y como PM debo escoger la que mejor se adapte a mi audiencia.

3.5. Enriqueciendo con recursos y anotaciones visuales

La tabla PERT no incluye recursos; este es un añadido que como PM hago durante la planificación. Para este proyecto, asignaré:

- `{Alice}` a las tareas de la ruta crítica y otras afines: A, C, E, H, I, J.
- `{Bob}` a las tareas de la rama secundaria: B, D, F, G, y también apoyará en H, I, J al 50%.

Estas asignaciones se traducen en PlantUML con la notación `on {Alice}` y `on {Bob:50%}`. El Gantt resultante mostrará los nombres de los recursos junto a las barras, permitiendo identificar rápidamente si Alice o Bob están sobrecargados. Si detecto que Bob tiene tres tareas solapándose, quizás deba secuenciar algunas o ajustar porcentajes. En la vida real, negociaría con él y con su líder funcional; en el plan, puedo simular distintos escenarios antes de comprometerme.

Añadiré también un código de colores para ayudar a la lectura:

- `is colored in LightBlue/Blue` para las tareas de Alice en la ruta crítica, destacando su importancia.
- `is colored in LightGreen/Green` para las tareas de Bob, mostrando la rama paralela.
- `is colored in Coral/Red` para el hito de integración y la tarea final J, que marcan entregas clave.

Finalmente, una nota textual resumirá la ruta crítica y la duración total, usando `note bottom ... end note`. Esto es especialmente útil para presentaciones a stakeholders que no necesitan los detalles internos de la red pero sí comprender el tiempo total y las actividades que no pueden retrasarse.

3.6. Visualizando la ruta crítica y las holguras en el Gantt

Un punto frecuentemente malinterpretado es cómo se manifiesta la ruta crítica en el Gantt. A diferencia del grafo PERT, donde la ruta crítica se resalta explícitamente, en el Gantt debemos inferirla. Las tareas de la ruta crítica (A, C, E, H, I, J) no tienen holgura; sus barras están "pegadas" una tras otra sin espacio para movimiento. Si una de ellas se desplaza, todo el proyecto se desplaza. En el diagrama que generaremos, esto se ve como una cadena continua de barras desde el inicio hasta el fin, sin huecos.

Por otro lado, las tareas con holgura (B, D, F, G) muestran visualmente espacios entre el final de una y el comienzo de la siguiente (por ejemplo, B termina, pero D podría empezar más tarde sin afectar la fecha final si Bob estuviera ocupado). Como líder técnico, debo vigilar la ruta crítica como un halcón, pero también mantener un ojo en las holguras, porque si B se retrasa demasiado (más de su holgura total, que en nuestro caso se calculó en la tabla PERT como 7 días para B, D, F y G), entonces la ruta B-D-G podría convertirse en la nueva ruta crítica y alterar la fecha de entrega.

PlantUML no calcula automáticamente las holguras (eso lo hicimos manualmente con CPM), pero al dibujar las dependencias tal cual, el motor de diseño sitúa cada barra lo más a la izquierda posible, respetando las restricciones. Por tanto, el Gantt resultante mostrará las fechas tempranas de inicio y fin, que corresponden a la planificación "todo empieza cuanto antes". Si quisiéramos mostrar las fechas tardías (lo más tarde que puede empezar una tarea sin retrasar el proyecto), necesitaríamos un diagrama de Gantt con "margen de demora", que PlantUML no soporta directamente pero podemos simular con anotaciones o usando un software de gestión de proyectos externo. No obstante, para los propósitos de esta clase, el Gantt temprano es más que suficiente para comunicar el plan y ejecutarlo.

3.7. El valor de la automatización: del Gantt manual al Gantt como código

Cierro esta sección con una reflexión. Hemos pasado de una tabla en PDF a un diagrama que se genera ejecutando un script. Si mañana el cliente pide añadir una actividad K entre H e I, solo tengo que editar el texto, ajustar las dependencias y regenerar la imagen. En cuestión de minutos, tengo un nuevo plan versionado. Si hubiéramos dibujado el Gantt a mano en una herramienta de arrastrar y soltar, cualquier cambio de este tipo implicaría reacomodar decenas de cajitas y conectores, con el consiguiente riesgo de error.

En la siguiente sección, tomaremos todo este diseño y lo codificaremos paso a paso en PlantUML, viendo cómo cada decisión tomada aquí se materializa en un diagrama profesional, listo para ser compartido con el equipo y los stakeholders. Recuerden: un buen Project Manager no solo sabe planificar, sino que sabe comunicar el plan de forma efectiva, y el Gantt generado con código es su mejor aliado.

4. Automatizando con PlantUML: del texto al Gantt profesional

Ahora que hemos sentado las bases teóricas y tenemos claro el diseño de nuestro proyecto, es el momento de convertir todo ese conocimiento en un artefacto real. Como Project Manager, valoro las herramientas que permiten automatizar la generación de diagramas porque eliminan la fricción del dibujo manual y establecen una única fuente de verdad. PlantUML, con su sintaxis declarativa inspirada en el

lenguaje natural, cumple exactamente ese propósito. En esta sección, les guiaré paso a paso en la construcción del diagrama de Gantt para el proyecto de 10 actividades, explicando cada línea de código y justificando las decisiones que tomo como líder del proyecto.

Antes de empezar, recuerden que todo el código que escriban debe ir dentro de un bloque delimitado por `@startgantt` y `@endgantt`, como se indica en la guía (página 341). Pueden ejecutar este código en su instalación local de PlantUML o en servicios en línea como [plantUML](#). Además, cada línea que empiece con una comilla simple (') es un comentario y no afecta al diagrama; es una buena práctica documentar el plan dentro del propio archivo fuente.

4.1. Paso 1: La columna vertebral del calendario y las tareas

Lo primero que debemos definir es la fecha de inicio del proyecto y los días no laborables. En el análisis habíamos decidido arrancar el martes 5 de mayo de 2026. Establecemos también que los fines de semana no se trabaja y añadimos un feriado el 15 de mayo para ver cómo se comporta el calendario. La sintaxis es:

```
Project starts 2026-05-05
saturday are closed
sunday are closed
2026-05-15 is closed
```

A continuación declaramos cada tarea con su duración. La notación `[A] requires 6 days` es la más directa (guía, página 341). Todas las duraciones están en días laborables, por lo que PlantUML omitirá automáticamente los sábados, domingos y el feriado al posicionar las barras. Es importante entender que `requires` define la duración de trabajo, no la duración calendario resultante.

Así, la declaración inicial de las diez tareas queda:

```
[A] requires 6 days
[B] requires 2 days
[C] requires 3 days
[D] requires 3 days
[E] requires 4 days
[F] requires 3 days
[G] requires 6 days
[H] requires 4 days
[I] requires 1 day
[J] requires 2 days
```

En este punto, si no añadiéramos dependencias, todas las tareas comenzarían el mismo día 5 de mayo (salvo que alguna estuviera restringida posteriormente). Eso no refleja la realidad, así que pasamos al siguiente nivel.

4.2. Paso 2: Tejiendo la red de dependencias

La esencia del Gantt está en cómo conectamos las tareas. Como vimos en la sección 3, la dependencia más común es fin-inicio, que en PlantUML se expresa con `starts at [Tarea]'s end`. Para tareas con una sola predecesora, la traducción es inmediata:

```
[C] starts at [A]'s end
[D] starts at [B]'s end
[E] starts at [C]'s end
[F] starts at [C]'s end
[G] starts at [D]'s end
[J] starts at [I]'s end
```

Para el caso de H, que depende tanto de E como de F, escribimos dos líneas:

```
[H] starts at [E]'s end
[H] starts at [F]'s end
```

El motor de diseño colocará H en la fecha más tardía entre los finales de E y F, que es exactamente lo que necesitamos.

La situación de I es más interesante. Debe comenzar cuando G y H hayan terminado. Sabemos que G está en la rama paralela que nace de B, y H en la rama crítica. Para obligar a I a esperar a ambas, usamos también dos líneas. Además, crearemos un hito explícito para señalar la convergencia, lo cual es una buena práctica de comunicación.

4.3. Paso 3: Insertando hitos para marcar sincronizaciones y entregables

Los hitos son eventos de duración cero. En nuestro plan, definimos `[Integración GH]` para indicar que ambas ramas se han unido. Este hito lo hacemos coincidir con el fin de H (la ruta crítica) y, al ponerlo antes de I, aseguramos que I no empiece hasta que también G esté lista (gracias a la dependencia `[I] starts at [G]'s end`). La sintaxis de hito es `happens at` o `happens` con una fecha (página 346). Aquí usamos la forma relativa:

```
[Integración GH] happens at [H]'s end
[I] starts at [Integración GH]'s end
[I] starts at [G]'s end
```

Con esto, I comenzará en el máximo entre el fin de G y el fin de H, modelando correctamente un “join” de dos ramas. Podríamos añadir más hitos si quisiéramos, por ejemplo `[Revisión de diseño]` al finalizar E, pero para mantener limpio el ejemplo nos centraremos en la convergencia principal.

4.4. Paso 4: Personalización visual para una comunicación efectiva

Un Gantt en blanco y negro es funcional, pero agregar colores, notas y etiquetas de recurso lo transforma en una herramienta de comunicación poderosa. Como PM, suelo asignar colores según la criticidad o el responsable. En este proyecto, destacaremos la ruta crítica y las tareas de cada miembro del equipo.

Recursos: Asignamos a Alice y Bob con la notación `on {Nombre}` y porcentajes de dedicación. Por ejemplo:

```
[A] on {Alice} requires 6 days
[B] on {Bob} requires 2 days
[C] on {Alice} requires 3 days
...
```

Como ya declaramos las tareas antes, podemos añadir los recursos en las mismas líneas o en líneas separadas. La sintaxis `[A] on {Alice} requires 6 days` es válida y compacta.

Colores: La cláusula `is colored in` acepta un color de fondo y uno de borde (o solo uno). Aplicaremos colores distintos para las tareas de Alice (en la ruta crítica) y Bob (rama secundaria). Por ejemplo:

```
[A] is colored in LightBlue/Blue
[B] is colored in LightGreen/Green
[C] is colored in LightYellow/Gold
[J] is colored in Coral/Red
```

También colorearemos el hito de integración para que resalte.

Nota explicativa: Al pie del diagrama, una nota resume la ruta crítica y la duración total. Se usa `note bottom ... end note`. Incluso dentro de la nota se puede usar formato Creole para resaltar texto (guía, página 15).

Línea vertical "hoy": Podemos marcar el día actual para seguimiento con `today is colored in #AAF` si quisiéramos, pero en esta fase de planificación inicial no es necesario. Lo menciono para que sepan que existe.

4.5. Código completo: el Gantt unificado

A continuación, presento el script completo que integra todos los elementos anteriores. Pueden copiarlo y ejecutarlo directamente en su entorno PlantUML. Observen cómo mantengo una estructura limpia con comentarios y secciones.

```
@startgantt
' Configuración del calendario
Project starts 2026-05-05
saturday are closed
sunday are closed
2026-05-15 is closed
```

```

' --- Definición de tareas con duraciones y recursos ---
[A] on {Alice} requires 6 days
[B] on {Bob} requires 2 days
[C] on {Alice} requires 3 days
[D] on {Bob} requires 3 days
[E] on {Alice} requires 4 days
[F] on {Bob} requires 3 days
[G] on {Bob} requires 6 days
[H] on {Alice} requires 4 days
[H] on {Bob:50%}
[I] on {Alice:50%} requires 1 day
[I] on {Bob:50%}
[J] on {Alice} requires 2 days
[J] on {Bob:50%}

' --- Dependencias simples ---
[C] starts at [A]'s end
[D] starts at [B]'s end
[E] starts at [C]'s end
[F] starts at [C]'s end
[G] starts at [D]'s end
[J] starts at [I]'s end

' --- Dependencia múltiple: H requiere E y F ---
[H] starts at [E]'s end
[H] starts at [F]'s end

' --- Hito de convergencia de ramas ---
[Integración GH] happens at [H]'s end
[I] starts at [Integración GH]'s end
[I] starts at [G]'s end

' --- Personalización de colores ---
[A] is colored in LightBlue/Blue
[B] is colored in LightGreen/Green
[C] is colored in LightYellow/Gold
[D] is colored in LightGreen/Green
[E] is colored in LightYellow/Gold
[F] is colored in LightGreen/Green
[G] is colored in LightGreen/Green
[H] is colored in LightBlue/Blue
[I] is colored in LightBlue/Blue
[J] is colored in Coral/Red
[Integración GH] is colored in Orange

' --- Nota con la ruta crítica ---
note bottom
  <b>Ruta crítica:</b> A → C → E → H → I → J
  <b>Duración total:</b> 20 días laborables
  <b>Feriado:</b> 15 de mayo no laborable
end note

@endgantt

```

Al ejecutarlo, obtendrán un diagrama de Gantt con todas las barras posicionadas dinámicamente según el calendario y las restricciones. Notarán que la barra de A inicia el 5 de mayo y termina el 12 de mayo (porque se saltó el fin de semana y el feriado no afecta porque está fuera de su rango). C, que depende de A, comienza el 13 de mayo y así sucesivamente. El hito de integración y la tarea J se situarán al final, mostrando claramente la fecha de finalización del proyecto.

4.6. Iteración y mejora continua: el Gantt como código vivo

Una vez generado el diagrama, lo revisamos con el equipo. Supongamos que Bob informa que no puede trabajar en G al 100% porque también debe atender otra incidencia. Modificamos su dedicación a `on {Bob:50%}` y vemos cómo la duración de G se extiende visualmente (aunque en PlantUML la duración sigue siendo 6 días, la barra se alarga en el tiempo calendario porque solo avanza la mitad por día). Si quisiéramos modelar esa extensión real, deberíamos aumentar la duración en días laborables, por ejemplo `[G] requires 12 days`. Este tipo de ajustes son inmediatos en código y nos permiten hacer simulaciones rápidas.

Otro escenario: el cliente solicita añadir una actividad K de 2 días después de H y antes de I. Basta con insertar `[K] requires 2 days` y modificar las dependencias: `[K] starts at [H]'s end`, `[I] starts at [K]'s end`, y eliminar la dependencia directa I→H. En menos de un minuto, tenemos un nuevo Gantt actualizado, sin riesgo de desordenar conectores manualmente.

Como cierre de esta práctica, les animo a versionar este archivo `.pum1` en un repositorio Git junto con el resto de la documentación del proyecto. Cada cambio en el plan quedará registrado en el historial, y podrán revisar quién modificó qué y por qué. Esta es la mentalidad de un Project Manager moderno: tratar el plan del proyecto como un activo de software más, sujeto a las mismas prácticas de calidad que el código fuente.

En la siguiente sección, pasaremos a un ejercicio práctico donde ustedes modificarán este Gantt añadiendo nuevos recursos, feriados y un hito adicional, consolidando así todo lo aprendido.

5. Ejercicio práctico: Evolucionando el Gantt como un Project Manager en escenarios reales

Hemos llegado al momento de la verdad. Ya conocen la teoría, analizaron el proyecto de 10 actividades y vieron cómo se traduce a un diagrama de Gantt automatizado con PlantUML. Ahora les propongo un ejercicio práctico que simula tres situaciones típicas que enfrentarás como líder de proyecto: la incorporación de una nueva integrante al equipo, un cambio en el calendario por un evento corporativo y la necesidad de agregar un punto formal de revisión de calidad. Su misión es modificar el Gantt original para adaptarlo a estos nuevos requerimientos, justificando cada decisión como lo harían frente a un comité de dirección.

Antes de comenzar, recuerden que en el entorno profesional el plan nunca es estático. Cambian las prioridades, se suman personas, surgen imprevistos. La habilidad de reaccionar rápidamente y actualizar el

cronograma de manera controlada es lo que distingue al Project Manager competente del que simplemente administra fechas. PlantUML, al ser código, les permite hacer estos cambios con precisión quirúrgica y tener trazabilidad completa mediante Git. Aprovechen ese poder.

5.1. Enunciado del ejercicio

Partiendo del diagrama de Gantt que construimos en la sección anterior (el proyecto de 20 días laborables con tareas A a J, ruta crítica A-C-E-H-I-J, inicio el 5 de mayo de 2026), realicen las siguientes modificaciones:

5.1.1. Cambio 1: Nueva integrante del equipo

Se incorpora **Carmen** como desarrolladora junior. El director de ingeniería le asigna las siguientes responsabilidades:

- Carmen trabajará al 100% en la tarea F (antes asignada a Bob).
- Carmen colaborará al 50% en la tarea H junto con Alice y Bob.
- Carmen apoyará al 50% en la tarea I y al 50% en la tarea J.

Reasignen los recursos según estas indicaciones. Reflexionen: ¿cambia la duración de alguna tarea? ¿Se altera la ruta crítica? ¿Bob queda con sobrecarga o con capacidad ociosa?

5.1.2. Cambio 2: Evento corporativo que bloquea el calendario

La empresa ha programado un **evento de team building** obligatorio para todo el equipo durante los días **20 y 21 de mayo de 2026**. Nadie trabajará esos días. Incorporen esta restricción en el calendario del Gantt. Determinen el impacto en la fecha de finalización del proyecto. ¿Cuántos días se desplaza la entrega? ¿Cambia la ruta crítica?

5.1.3. Cambio 3: Hito de revisión de calidad

El departamento de QA exige un **punto de control formal** después de que terminen las tareas E y F, pero antes de que comience H. Este hito, que llamaremos **[Revisión QA intermedia]**, tendrá una duración de 0 días pero debe aparecer claramente en el diagrama como un diamante (la representación por defecto de los hitos). Además, la nota al pie del Gantt debe actualizarse para reflejar este nuevo hito y cualquier cambio en la ruta crítica o la duración total.

5.2. Guía paso a paso para resolver el ejercicio

Les sugiero abordar los cambios en orden, regenerando el diagrama después de cada uno para verificar visualmente el impacto. También pueden hacer todos los cambios de una vez y luego analizar el resultado final, pero es más pedagógico iterar.

5.2.1. Solución del Cambio 1: Incorporación de Carmen

Primero, modificamos las declaraciones de recursos en las tareas afectadas. La tarea F originalmente era **[F] on {Bob} requires 3 days**. Ahora será **[F] on {Carmen} requires 3 days**. La duración no cambia porque Carmen trabajará al 100% y la complejidad de F no se modifica.

Para H, la declaración anterior era:

```
[H] on {Alice} requires 4 days  
[H] on {Bob:50%}
```

Añadimos a Carmen al 50%. Dado que Alice ya está al 100% y Bob y Carmen al 50% cada uno, la capacidad total asignada a H sería del 200%, lo cual es suficiente para mantener la duración de 4 días sin problemas. Sin embargo, en la vida real debo verificar que la suma de dedicaciones no supere la capacidad real del equipo en ese período; en este caso, 200% significa dos personas equivalentes a tiempo completo, lo cual es razonable.

Para I, teníamos:

```
[I] on {Alice:50%} requires 1 day  
[I] on {Bob:50%}
```

Añadimos **[I] on {Carmen:50%}**. La dedicación total ahora es 150% para una tarea de 1 día, lo que está más que cubierto.

Para J, análogamente:

```
[J] on {Alice} requires 2 days  
[J] on {Bob:50%}
```

Añadimos **[J] on {Carmen:50%}**.

También debo declarar a Carmen como recurso disponible. En PlantUML no es necesario declarar los recursos explícitamente, pero es una buena práctica documentarlo en un comentario.

Impacto en la ruta crítica: Las tareas de la ruta crítica (A, C, E, H, I, J) mantienen sus duraciones. La ruta crítica original sigue siendo la misma. Bob ahora tiene menos carga (pierde F y comparte H, I, J), lo cual podría permitirle asumir otras tareas o simplemente tener un buffer de capacidad. Como PM, debo revisar si Bob queda subutilizado y, de ser así, reasignarlo a otras tareas del proyecto o a otro proyecto.

5.2.2. Solución del Cambio 2: Evento corporativo

Simplemente añadimos la línea:

```
2026-05-20 to 2026-05-21 is closed
```

Al hacerlo, cualquier tarea que estuviera activa durante esos dos días se alargará automáticamente. Debemos identificar qué tareas se ven afectadas. Revisemos el cronograma original:

- A: termina el 12 de mayo (no afectada).
- B: termina el 6 de mayo (no afectada).
- C: empieza el 13 de mayo y termina el 15 de mayo (pero el 15 es feriado, así que C en realidad iría del 13 al 18 de mayo saltando el 15, 16 y 17 son fin de semana; C terminaría el 18 de mayo si no hay más feriados... hay que recalcular con cuidado).
- D: empieza el 7 de mayo y termina el 11 de mayo (no afectada).
- E: empieza tras C (19 de mayo) y dura 4 días: 19, 20, 21, 22 de mayo. Pero 20 y 21 son los días del evento, por lo que E se extiende hasta el 26 de mayo (saltando fin de semana 23-24).
- F: empieza junto con E (19 de mayo) y dura 3 días: originalmente 19, 20, 21, pero con el evento, se va a 19, 22, 25 de mayo.
- G: empieza tras D (12 de mayo) y dura 6 días: terminaría el 19 de mayo (no afectada directamente, aunque si G se retrasase por otra razón podría afectar; en principio G escapa del evento).
- H: depende de E y F. Con E terminando el 26 de mayo y F terminando el 25 de mayo, H comenzará el 27 de mayo (máximo de los finales). Su duración de 4 días la llevaría hasta el 1 de junio.
- I: comienza tras H y G. G terminó el 19 de mayo, H termina el 1 de junio, por lo que I empieza el 2 de junio. Dura 1 día, termina el 2 de junio.
- J: comienza el 3 de junio, dura 2 días, termina el 4 de junio.

Por lo tanto, la fecha de finalización se desplaza del 28 de mayo (fecha original sin el evento) al 4 de junio. La ruta crítica sigue siendo A-C-E-H-I-J, pero su duración total ahora es mayor debido al parón del evento. ¡El proyecto se retrasa aproximadamente una semana!

5.2.3. Solución del Cambio 3: Hito de revisión de calidad

Agregamos el hito justo después de los finales de E y F. La manera más limpia es:

```
[Revisión QA intermedia] happens at [E]'s end
[Revisión QA intermedia] happens at [F]'s end
```

Luego, H debe esperar a que este hito ocurra (en lugar de depender directamente de E y F). Modificamos:

```
[H] starts at [Revisión QA intermedia]'s end
```

Y eliminamos las líneas `[H] starts at [E]'s end` y `[H] starts at [F]'s end`. Esto asegura que H no comience hasta que ambas tareas hayan sido revisadas, lo cual puede implicar una pequeña espera administrativa aunque el hito tenga duración cero.

Finalmente, actualizamos la nota al pie para reflejar la nueva ruta crítica (que sigue siendo la misma pero incluyendo el hito) y la nueva duración total. Por ejemplo:

```
note bottom
  <b>Ruta crítica:</b> A → C → E → Revisión QA → H → I → J
  <b>Duración total:</b> 23 días laborables (aprox.)
  <b>Feriados:</b> 15 de mayo, 20-21 de mayo (team building)
```

```
<b>Recursos:</b> Alice, Bob, Carmen  
end note
```

5.3. Solución completa del ejercicio

A continuación, les entrego el código PlantUML que integra los tres cambios. Podrán compararlo con el original y ver cómo evolucionó el plan. Analícenlo línea por línea y ejecútenlo para obtener el nuevo diagrama.

```
@startgantt  
  ' Configuración del calendario extendida  
  Project starts 2026-05-05  
  saturday are closed  
  sunday are closed  
  2026-05-15 is closed  
  2026-05-20 to 2026-05-21 is closed  
  
  ' --- Tareas con recursos actualizados ---  
  [A] on {Alice} requires 6 days  
  [B] on {Bob} requires 2 days  
  [C] on {Alice} requires 3 days  
  [D] on {Bob} requires 3 days  
  [E] on {Alice} requires 4 days  
  [F] on {Carmen} requires 3 days  
  [G] on {Bob} requires 6 days  
  [H] on {Alice} requires 4 days  
  [H] on {Bob:50%}  
  [H] on {Carmen:50%}  
  [I] on {Alice:50%} requires 1 day  
  [I] on {Bob:50%}  
  [I] on {Carmen:50%}  
  [J] on {Alice} requires 2 days  
  [J] on {Bob:50%}  
  [J] on {Carmen:50%}  
  
  ' --- Dependencias simples ---  
  [C] starts at [A]'s end  
  [D] starts at [B]'s end  
  [E] starts at [C]'s end  
  [F] starts at [C]'s end  
  [G] starts at [D]'s end  
  [J] starts at [I]'s end  
  
  ' --- Hito de revisión de calidad ---  
  [Revisión QA intermedia] happens at [E]'s end  
  [Revisión QA intermedia] happens at [F]'s end  
  
  ' --- H ahora depende del hito en lugar de E y F directamente ---  
  [H] starts at [Revisión QA intermedia]'s end
```

```

' --- Hito de convergencia de ramas ---
[Integración GH] happens at [H]'s end
[I] starts at [Integración GH]'s end
[I] starts at [G]'s end

' --- Personalización de colores ---
[A] is colored in LightBlue/Blue
[B] is colored in LightGreen/Green
[C] is colored in LightYellow/Gold
[D] is colored in LightGreen/Green
[E] is colored in LightYellow/Gold
[F] is colored in LightGreen/Green
[G] is colored in LightGreen/Green
[H] is colored in LightBlue/Blue
[I] is colored in LightBlue/Blue
[J] is colored in Coral/Red
[Revisión QA intermedia] is colored in Fuchsia
[Integración GH] is colored in Orange

' --- Nota actualizada ---
note bottom
  <b>Ruta crítica:</b> A → C → E → Revisión QA → H → I → J
  <b>Duración total:</b> 23 días laborables
  <b>Feriados:</b> 15 de mayo, 20-21 de mayo (team building)
  <b>Equipo:</b> Alice, Bob, Carmen
end note

@endgantt

```

5.4. Preguntas de reflexión y cierre del ejercicio

Después de implementar los cambios, quiero que reflexionen sobre estas cuestiones, ya que son las que me han hecho en reuniones reales con stakeholders:

1. **Carga de trabajo:** ¿Algún recurso quedó sobreasignado (más del 100% en algún período)?
Observen que Bob ahora comparte H, I, J con Carmen al 50%, pero antes ya estaba al 50% en H y al 100% en otras; con la redistribución, ¿su carga total es sostenible? ¿Debería rebalancearse?
2. **Impacto del evento:** ¿Qué habría pasado si el team building hubiera caído justo en medio de la tarea A (ruta crítica)? ¿Cómo mitigarían ese riesgo en un proyecto real? Una respuesta podría ser negociar con RRHH para que el equipo del proyecto quede exento, o planificar el proyecto con un buffer de contingencia.
3. **Comunicación visual:** ¿El diagrama resultante es claro para alguien que no conoce el proyecto? ¿Los hitos están bien diferenciados? ¿Los colores ayudan o distraen? En mi experiencia, a veces menos es más: demasiados colores pueden confundir. Evalúen si la paleta elegida transmite correctamente la prioridad de las tareas.
4. **Gestión del cambio:** Si mañana el cliente pide reducir la duración total a 18 días laborables, ¿qué acciones tomarían? Piensen en alternativas como añadir más recursos a la ruta crítica (¿se puede

paralelizar algo?), recortar alcance (¿realmente se necesita QA intermedia?), o negociar horas extra. El Gantt les permite simular estas opciones modificando duraciones y viendo el efecto inmediato.

Como cierre, les pido que versionen este nuevo archivo en un repositorio Git, con un mensaje de commit descriptivo como "feat: incorporar a Carmen, evento team building 20-21 mayo, hito QA intermedia". Con eso, habrán completado el ciclo completo de planificación, ejecución simulada y adaptación a cambios, todo ello con una herramienta profesional y trazable. En la próxima sesión, abordaremos cómo integrar este Gantt con otros diagramas UML (casos de uso, clases, secuencia) para tener una documentación de proyecto completa y coherente.

¡Buen trabajo, futuros Project Managers! El Gantt ya no tiene secretos para ustedes.

6. Buenas prácticas

Hemos recorrido un camino intenso. Partimos desde los fundamentos teóricos del diagrama de Gantt, los conectamos con el análisis PERT/CPM de un proyecto real de 10 actividades, desmenuzamos cada uno de los elementos que componen un cronograma profesional, traducimos todo ese diseño a código PlantUML y, finalmente, enfrentamos un ejercicio práctico que simulaba las turbulencias típicas de un proyecto de software: cambios de personal, eventos corporativos y exigencias de calidad. Ahora, en esta última sesión, quiero destilar las lecciones aprendidas en una serie de buenas prácticas que llevarán con ustedes a lo largo de su carrera como ingenieros de software y futuros líderes técnicos. No se trata solo de saber usar una herramienta; se trata de interiorizar una mentalidad de gestión.

6.1. Lo que hemos construido

Retrocedamos un instante. Empezamos con una tabla en un PDF que contenía actividades, duraciones estimadas con PERT y precedencias. Ese era nuestro dato crudo. Luego, lo proyectamos sobre un calendario real, añadimos recursos, hitos y restricciones, y finalmente lo codificamos en un lenguaje declarativo (PlantUML) que genera automáticamente el diagrama. Este flujo —dato, modelo, código, visualización— es el mismo que usamos en la ingeniería de software para construir sistemas. Como Project Manager, ustedes no serán simples dibujantes de barras; serán arquitectos de la dimensión temporal del proyecto, capaces de modelar, simular y comunicar la planificación con la misma precisión con la que un desarrollador modela una base de datos.

El diagrama resultante no es un adorno. Es un contrato visual con el equipo y los stakeholders. Cada barra, cada hito, cada dependencia cuenta una historia: quién hace qué, cuándo, y qué pasa si algo se mueve. Y como todo buen contrato, debe ser claro, inequívoco y mantenible. De eso tratan las buenas prácticas que compartiré a continuación.

6.2. Buenas prácticas para la gestión del Gantt con PlantUML

6.2.1. El Gantt es código: trátalo como tal

Si hay una idea que quiero que graben a fuego, es esta: el archivo `.puml` donde definen el cronograma es **código fuente**, igual que un `.java`, un `.py` o un `.sql`. Como tal, merece las mismas prácticas de calidad:

- **Control de versiones:** Almacénalo en Git junto al resto del proyecto. Cada modificación de fechas, recursos o dependencias genera un commit con su respectivo mensaje descriptivo. Si alguien pregunta por qué la tarea H ahora depende de un hito de QA, pueden rastrear el cambio hasta el commit donde se introdujo esa restricción.
- **Revisiones entre pares:** ¿Modificaron la ruta crítica? Abran un Pull Request y pidan a otro ingeniero que revise el cambio. Una segunda mirada puede detectar una dependencia cíclica, un recurso sobreasignado o una fecha poco realista.
- **Integración continua:** Pueden configurar un pipeline que, al hacer push, regenere la imagen del Gantt y la publique en un wiki, en Confluence o en la documentación del proyecto. Así, el diagrama siempre está actualizado sin intervención manual.
- **No repitas código (DRY):** Si tienen tareas que se repiten en varios proyectos, encapsúlenlas en procedimientos reutilizables con `!procedure` y `!include`. La guía de PlantUML (páginas 518-523) explica cómo definir funciones y reutilizar archivos. Como ingenieros, deben evitar la duplicación a toda costa.

6.2.2. Calendario realista: el tiempo no es infinito ni uniforme

Un error clásico de principiante es asumir que todos los días son laborables y que el equipo está disponible al 100% todo el tiempo. La realidad es tozuda: fines de semana, feriados, vacaciones, eventos corporativos, pausas por formación, bajas médicas. Si no modelan estas restricciones en el calendario, el Gantt será una fantasía.

En nuestro ejercicio, añadir dos días de team building desplazó la fecha de entrega en casi una semana. Ese es el tipo de impacto que los stakeholders necesitan conocer cuanto antes. Como Project Manager, mantengan un calendario actualizado y, si es posible, negocien con la organización para proteger los períodos críticos del proyecto. Recuerden que en PlantUML pueden definir días cerrados con mucha precisión: un solo día, un rango, o incluso patrones como `saturday are closed`.

6.2.3. Recursos: sin personas, el plan es papel mojado

Asignar recursos no es opcional. Un Gantt que solo muestra tareas sin responsables es como un plano sin obreros: bonito, pero inútil para ejecutar. Cuando asignan `on {Alice}`, `on {Bob:50%}`, están respondiendo a la pregunta más básica de cualquier miembro del equipo: “¿qué se supone que debo hacer esta semana?”.

Pero además, la asignación de recursos revela cuellos de botella ocultos. En el ejercicio, vimos que redistribuir a Carmen alivió a Bob, pero también pudimos haber detectado que Alice está sobrecargada porque aparece en casi todas las tareas de la ruta crítica. Como PM, debo preguntarme: ¿Alice es realmente la única que puede hacer esas tareas? ¿Puedo capacitarla en otras o transferir conocimiento para reducir el riesgo de dependencia? El Gantt no da esas respuestas, pero las visibiliza.

PlantUML también permite ocultar recursos (`hide resources names`) para audiencias externas. Usen ese poder con criterio: un cliente no necesita saber que Alice trabaja al 50% en H; solo necesita saber que H se completa en la fecha prevista.

6.2.4. Hitos como faros de gobernanza

Los hitos son sus aliados para la comunicación con la dirección. Un comité de seguimiento no quiere revisar un mar de barras; quiere ver puntos de control claros: “¿El diseño está aprobado? ¿La integración se completó? ¿Las pruebas de aceptación pasaron?”.

Acostúmbrense a definir hitos en las fronteras de fase y en los puntos de sincronización entre equipos. En nuestro proyecto, [Revisión QA intermedia] e [Integración GH] son ejemplos perfectos: marcan transiciones importantes y fuerzan una pausa para verificar calidad antes de continuar. Además, los hitos son binarios (se cumplen o no), lo que facilita el seguimiento con indicadores objetivos.

6.2.5. Comunicación visual efectiva: colores, notas y leyendas

Un Gantt es un artefacto de comunicación. Si el diagrama es confuso, el mensaje se pierde. Algunas reglas que aplico:

- **Colores con propósito:** No pinten por pintar. Usen una paleta consistente (por ejemplo, azul para la ruta crítica, verde para tareas de soporte, rojo para hitos). Si todo es de colores chillones, nada destaca.
- **Notas explicativas:** Una *note bottom* con la ruta crítica y la duración total es increíblemente útil para quien ve el diagrama sin contexto.
- **Leyendas:** Si usan muchos símbolos o colores, incluyan una *legend* que los explique. La guía de PlantUML lo cubre en la sección de comandos comunes (páginas 419-420).

Recuerden que el diagrama puede ser incrustado en documentos más amplios (Markdown, HTML, PDF). Asegúrense de que la imagen generada tenga la resolución adecuada y que el texto sea legible. Pueden usar *scale* o *skinparam dpi* para ajustar el tamaño.

6.3. Errores frecuentes y cómo evitarlos

En mi experiencia supervisando a ingenieros novatos en la gestión de proyectos, he visto estos tropiezos una y otra vez. Comparto los más comunes para que los eviten:

1. **Sobreplanificar al inicio y nunca actualizar:** Pasan tres días definiendo el Gantt perfecto, lo guardan en un PDF y nunca más lo miran. El proyecto avanza y el Gantt queda obsoleto. Solución: integren la actualización del Gantt en la rutina semanal (por ejemplo, al cierre del sprint).
2. **Confundir duración con esfuerzo:** Ya lo mencioné antes, pero es tan frecuente que lo repito. Si una tarea requiere 40 horas de esfuerzo y asignan dos personas, la duración puede ser 2.5 días, no 5. Si no ajustan la duración en el Gantt al añadir recursos, el cronograma será incorrecto.
3. **Dependencias demasiado rígidas:** No todo es fin-inicio. A veces dos tareas pueden solaparse (inicio-inicio) o una puede empezar antes de que termine la anterior si tiene un avance parcial. Modelen esas situaciones con *starts at [Tarea]'s start* o con hitos intermedios.
4. **Ignorar las holguras:** Que una tarea tenga holgura no significa que sea irrelevante. Si B se retrasa más de lo permitido, se convierte en crítica. Monitoreen las holguras como quien vigila el nivel de batería del teléfono: mientras hay, no hay alarma, pero si se agota, entramos en pánico.
5. **No comunicar los cambios:** Modifican el Gantt en silencio y esperan que todos se enteren. Como PM, cada cambio en el cronograma debe ir acompañado de una comunicación clara al equipo y a

los stakeholders, explicando el impacto y las medidas tomadas.

6.4. El siguiente paso: integración con la documentación UML completa

El diagrama de Gantt no vive aislado. En un proyecto real, forma parte de un ecosistema de modelos que incluyen:

- **Casos de uso:** Definen qué funcionalidades se entregan. El Gantt muestra cuándo se implementa cada una.
- **Diagramas de clases:** Definen la estructura del software. Pueden usar el Gantt para planificar la construcción de los módulos.
- **Diagramas de secuencia:** Definen interacciones. El Gantt puede reflejar el orden en que se prueban esos escenarios.
- **Diagramas de componentes y despliegue:** Definen la arquitectura física. El Gantt incluye tareas de instalación, configuración y puesta en marcha.

PlantUML brilla aquí porque unifica todos estos diagramas bajo un mismo lenguaje y repositorio. Pueden tener un archivo `plan.puml` con el Gantt, `casos.puml` con los casos de uso, `clases.puml` con el modelo de dominio, y referenciarlos entre sí mediante `!include`. Incluso pueden incrustar fragmentos de Gantt dentro de un diagrama de actividades para mostrar la planificación temporal de un proceso.

Los invito a explorar la documentación de PlantUML en las secciones que no cubrimos en clase: diagramas de secuencia avanzados, preprocesador, funciones, temas y la integración con JSON/YAML. El documento que tienen ([PlantUML_Language_Reference_Guide_es.pdf]) es una mina de oro que les servirá como referencia continua.

6.5. Reflexión final: el Project Manager ingeniero

Durante esta clase, he insistido en que ustedes no son simplemente “gestores”, son **ingenieros de software que gestionan**. Esa diferencia es fundamental. Un gestor tradicional llena hojas de cálculo; un ingeniero-gestor modela, automatiza, versiona y optimiza. El Gantt que construimos es una prueba tangible de esa mentalidad: tomaron un problema de planificación, lo descompusieron en elementos atómicos, lo codificaron, lo validaron visualmente, lo sometieron a cambios y lo versionaron. Es exactamente el mismo ciclo que aplican al desarrollar software.

Cuando dentro de unos años lideren su primer proyecto, recuerden esta clase. Abran su editor de código, escriban `@startgantt`, y empiecen a teclear con confianza. Porque el mejor diagrama de Gantt no es el más bonito, sino el que usa el equipo cada día para saber dónde está, hacia dónde va, y qué obstáculos hay en el camino. Y ese diagrama, ahora, está bajo su control.

Confíen en el método, confíen en las herramientas, pero sobre todo, confíen en su capacidad para aprender y adaptarse. La ingeniería del software es una disciplina de cambio constante; quienes dominan la planificación flexible sobreviven y prosperan. Ustedes ya dieron el primer paso.

Gracias por su atención y compromiso. Ahora, a planificar proyectos como los campeones que son.